Costanza Incostante

DISIM - Università degli Studi dell'Aquila

Street Science 2025

Problema

La professoressa Costanza Stefanini, decana di UnivAQ, partecipa a una conferenza atipica. La manifestazione si compone di $n \in \mathbb{N}^+$ eventi, ciascuno con il proprio prestigio $a_i \in \mathbb{N}^+, \quad \forall i \in \{1, \dots n\}$. L'Intelligenza Artificiale le consiglia, stranamente, di selezionare solo insiemi di eventi "senza buchi", ovvero $a_i, a_{i+1}, \dots, a_j \quad i, j \in \mathbb{N}^+ : i \leq j$. Inoltre, per scaramanzia, un sottoinsieme di eventi è ammissibile se e solo se la somma di tutti i prestigi è pari. Formalmente, $a_i + a_{i+1} + \dots + a_j = 2k, \quad k \in \mathbb{N}^+$. Aiuta la professoressa a **contare** tutti le possibili sotto-sequenze di eventi ammissibili.

INPUT

- $n \in \mathbb{N}^+$;
- $(a_1, \dots, a_n) : a_i \in \mathbb{N}^+$.

OUTPUT

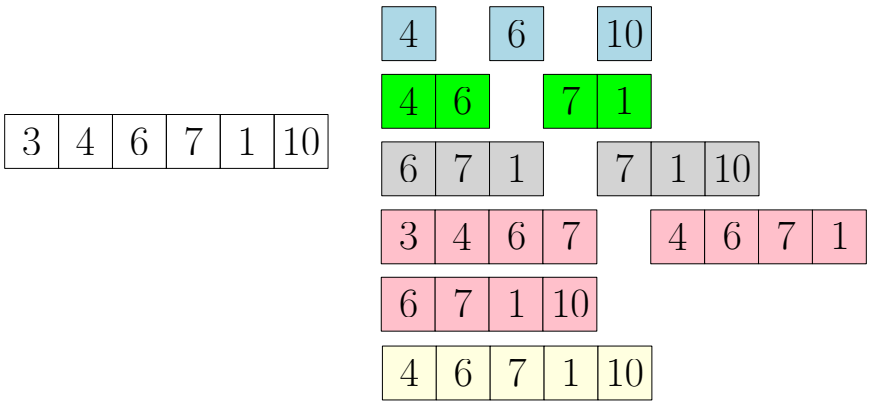
- il *numero* P di sotto-sequenze di elementi contigui aventi somma pari.

VINCOLI

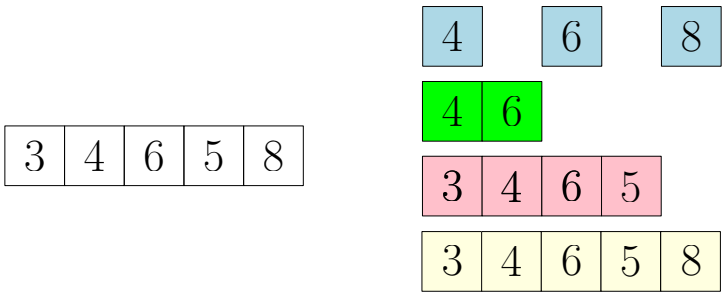
- $1 \leq a_i \leq$ INT_MAX;
- $2 \leq n \leq 10^9$;
- **Limite di tempo**: 2 secondi;
- **Limite di memoria**: 256 Mb.

Esempi

In questo primo esempio sono elencate tutte le sotto-sequenze di somma pari.



Answer: 11



Answer: 6

Costanza Incostante - Soluzioni

Giordano Colli, Riccardo Marcone

1 Forza Bruta $O(n^3)$

Idea: Considerare ogni sottosequenza la cui somma degli elementi è pari: (Soluzione esaustiva o Brute Force).
Data una sequenza A, una sua sottosequenza $(a_i, \ldots, a_j)$ è univocamente rappresentata dalla tupla $(i, j) : i \leq j$.

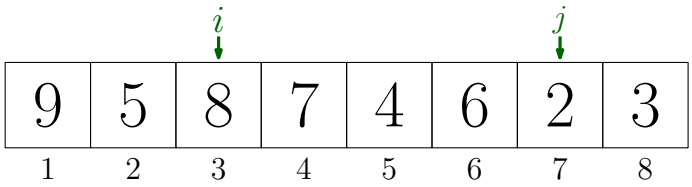


Figure 1: Nell'immagine e' mostrata una sequenza A di 8 elementi e gli indici $i = 3$ e $j = 7$, rappresentanti la sottosequenza $(a_3, a_4, a_5, a_6, a_7)$.

Segue un algoritmo che considera la somma di ogni possibile sottosequenza, fissando iterativamente indici di inizio (i) e fine (j).

Algorithm 1: BruteForce(A)

```
1 SottoSequenzePari ← 0;
2 for i ← 1 to n do
3     for j ← i to n do
4         S ← Σ_{k=i}^{j} a_k ;
5         if S è pari then
6             SottoSequenzePari ← SottoSequenzePari +1;
7 return SottoSequenzePari;
```

1.1 Correttezza

Dato che alla sottosequenza $(a_i, \ldots, a_j) : 1 \leq i \leq j \leq n$ corrisponde la tupla (i, j) e viceversa, i due cicli `for` alle linee 2 e 3 garantiscono che ogni sottosequenza venga considerata, mentre l'`if` a riga 5 garantisce che la sottosequenza in essere venga contata se e solo se questa ha somma (S) pari.

1.2 Complessità Computazionale

Lemma 1.1. *Il numero di sottosequenze relativo ad una sequenza di $n \in \mathbb{N}^+$ elementi è $\Theta(n^2)$.*

Proof. Il numero di sottosequenze di lunghezza $k \in \mathbb{N}^+ \quad 1 \leq k \leq n \quad$ è uguale a $n - k + 1$. Data la lunghezza k della sottosequenza, j deve essere uguale a $i + k - 1$ e, se la sottosequenza è ammissibile, $j \leq n$. Dunque $i + k - 1 = j \leq n$, altrimenti sarebbe stato scelto un indice j al di fuori della sequenza. La precedente implica direttamente $i \leq n - k + 1$ ed il numero di indici i di inizio di una qualunque

sottosequenza di lunghezza k è $\leq n-k+1$, e in particolare ogni sottosequenza con $i \in \{1, \dots, n-k+1\}$ e $j = i+k-1$ è valida.
Quindi, il numero totale di sottosequenze in una sequenza di lunghezza n è:

$$\sum_{k=1}^{n}(n-k+1) = \sum_{i=1}^{n} i \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} i \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} \left\lceil \frac{n}{2} \right\rceil \geq \frac{(n-1)}{2}\frac{(n-1)}{2} \in \Omega(n^2)$$

Osserva inoltre che le sottosequenze sono $O(n^2)$ dato che il numero delle stesse è limitato superiormente dal numero di coppie $|\{(i,j) : i,j \in \{1,\dots,n\}\}| = n^2 \in O(n^2)$. □

Inoltre si osserva che il numero di sottosequenze in una sequenza di $n \in \mathbb{N}^+$ e' esattamente la somma di Gauss da 1 ad n, ovvero $\displaystyle\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

Lemma 1.2. *Sia A una sequenza di $n \in \mathbb{N}^+ : n \geq 3$ elementi, esistono $\Omega(n^2)$ sottosequenze di lunghezza $\Omega(n)$.*

Proof. Sia $A_1 = \{1, \dots, \lfloor \frac{n}{3} \rfloor\}$ e $A_2 = \{n - \lfloor \frac{n}{3} \rfloor, \dots n\}$, si osservi che qualsiasi sottosequenza con indice di inizio in A_1 e indice di fine in A_2 contiene almeno $\lfloor \frac{n}{3} \rfloor$ elementi compresi tra $\lfloor \frac{n}{3} \rfloor + 1$ ed $n - \lfloor \frac{n}{3} \rfloor - 1$. Il numero di sottosequenze con indice di inizio in A_1 e indice di fine in A_2 è uguale ad $|A_1| \cdot |A_2| \in \Theta(\frac{n}{3} \cdot \frac{n}{3}) \subseteq \Theta(n^2)$, ognuna di queste sequenze ha lunghezza $\Theta(\frac{n}{3}) \subseteq \Theta(n) \subseteq \Omega(n)$, da cui segue il lemma. □

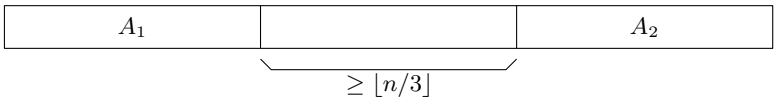


Figure 2: L'immagine rappresenta le precedentemente descritte parti A_1 e A_2 in una sequenza A di n elementi.

Theorem 1.3. *L'algoritmo `BruteForce` ha complessità computazionale superiormente limitata da una funzione in $\Theta(n^3)$, dove $|A| = n$.*

Per ogni sottosequenza (i,j) fissata, l'algoritmo computa la somma $\displaystyle\sum_{k=i}^{j} a_k$, richiedendo $\Theta(l)$ istruzioni, dove $l = j - i + 1$ rappresenta la lunghezza della sequenza (i,j). Dal Lemma 1.2 segue che l'algoritmo ha complessitá computazionale $\Omega(n^3)$, dato che ci sono $\Theta(n^2)$ sottosequenze, la cui lunghezza è $\Omega(n)$. Inoltre non vengono eseguite $\omega(n^3)$ istruzioni, dato che dal Lemma 1.1 ci sono $O(n^2)$ sottosequenze, e chiaramente tutte hanno lunghezza $O(n)$.

2 Somme prefisse $O(n^2)$

Idea: Precalcolare $S(i,j) = \displaystyle\sum_{k=i}^{j} a_k$.
Nell'algoritmo `BruteForce`, le istruzioni utilizzate per ricalcolare la somma degli elementi di ciascuna sottosequenza possono essere risparmiate.
Calcolando la somma degli elementi di ogni sottosequenza che ha come indice di inizio 1, è possibile calcolare velocemente(in tempo $O(1)$) qualunque sottosequenza rappresentata da $(i,j) : 1 \leq i \leq j \leq n$.
Sia $S(j) = \displaystyle\sum_{k=1}^{j} a_k$, $S(i,j) = \displaystyle\sum_{k=i}^{j} a_k$ viene rappresentato mediante l'utilizzo di $S(i-1) = \displaystyle\sum_{k=1}^{i-1} a_k$ e $S(j) = \displaystyle\sum_{k=1}^{j} a_k$.

$$S(i,j) = S(j) - S(i-1) \quad 1 < i \leq j \leq n$$

Se $i = 1$, allora $S(i,j) = S(1,j) = S(j)$, precedentemente calcolato.
Per evitare di verificare se $i = 1$, viene utilizzato $S(0) = 0$ e la formula precedente diventa consistente

per $1 \leq i \leq j \leq n$.

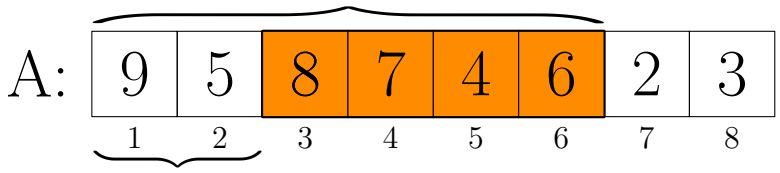


Figure 3: Nell'immagine viene rappresentata l'identita' $S(3,6) = S(6) - S(2)$

Nell'algoritmo che segue viene calcolato $S(i) \quad \forall i \in \{1, \ldots, n\}$ sfruttando la seguente proprietà: $S(i) = S(i-1) + a_i$.

Algorithm 2: PrefixSumBuild(A)

```
S(0) ← 0;
for i ← 1 to n do
    S(i) ← S(i − 1) + a_i;
return S;
```

Di seguito, una rappresentazione della sequenza delle somme prefisse calcolate a partire dalla sequenza A raffigurata:

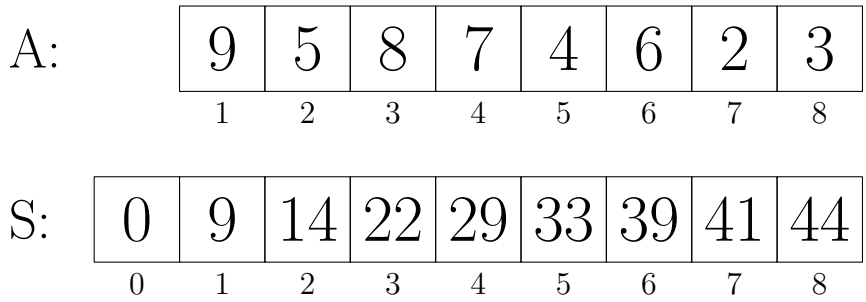


Figure 4: Esempio: $S(6) = S(5) + a_6 = 33 + 6 = 39$

Di seguito l'algoritmo a forza bruta, che ricalcola velocemente somme di elementi di sottosequenze, utilizzando le **somme prefisse**.

Algorithm 3: BruteForce++(A)

```
SottoSequenzePari ← 0;
S ← PrefixSumBuild(A) ;
for i ← 1 to n do
    for j ← i to n do
        Sum ← S(j) − S(i − 1) ;
        if Sum è pari then
            SottoSequenzePari ← SottoSequenzePari +1;
return SottoSequenzePari;
```

2.1 Correttezza

Lemma 2.1. *Sia A una sequenza di n elementi, l'algoritmo* `PrefixSumBuild` *calcola correttamente* $S(j) = \sum_{k=1}^{j} a_k \quad \forall j \in \{1, \ldots, n\}$ *utilizzando $O(n)$ istruzioni.*

Proof. Dalla riga 1, $S(0) = 0$.
Supponiamo induttivamente che nel momento in cui $S(i)$ viene calcolato $S(i-1)$ sia stato correttamente determinato. Dalla riga 3 in `PrefixSumBuild`, $S(i) = S(i-1) + a_i = \sum_{k=1}^{i-1} a_k + a_i = \sum_{k=1}^{i} a_k$.
Si osservi inoltre che $\Theta(n)$ istruzioni vengono eseguite dall'algoritmo. □

Usando il Lemma 2.1,è possibile affermare che ogni somma relativa ad ogni sottosequenza è correttamente determinata, pertanto vale la prova di correttezza utilizzata per l'algoritmo `BruteForce`.

2.2 Complessità Computazionale

Theorem 2.2. *Sia A una sequenza di n elementi, l'algoritmo `BruteForce++` ha complessità computazionale superiormente limitata da una funzione in $\Theta(n^2)$, dove $|A| = n$.*

Proof. Dal Lemma 2.1 il calcolo delle somme prefissa a riga 2 richiede $O(n)$ istruzioni. Dal Lemma 1.1, $\Theta(n^2)$ sottosequenze vengono considerate, e per ognuna di esse, $O(1)$ istruzioni vengono utilizzate nelle righe $5, 6, 7$. In totale $\Theta(n^2)$ istruzioni vengono impiegate dall'algoritmo. □

3 Ottimo $O(n)$

Idea: È possibile ridursi a considerare le coppie di elementi all'interno della sequenza di somme prefisse. In particolare, ogni sottosequenza di A ha somma dei propri elementi rappresentabile attraverso la differenza di una coppia di elementi nelle somme prefisse.
Questa idea si sviluppa scandendo tutte le coppie $(i, j) : 0 \leq i < j \leq n$ nella sequenza delle somme prefisse, verificando quale coppia ha differenza pari, ovvero:

$$S(i, j) = S(j) - S(i-1) \text{ è pari.} \iff S(i-1) \text{ e } S(j) \text{ sono entrambi pari(dispari).}$$

Segue che fissato j, il numero di sottosequenze terminanti in j con somma pari è equivalente al numero di indici $i < j : S(i)$ e $S(j)$ hanno la stessa parità. Siano ora definiti i seguenti insiemi di indici:

$P = \{i : S(i) \text{ è pari}\}$

$D = \{i : S(i) \text{ è dispari}\}$

Allora, ogni coppia (i, j) con $i < j$ tale che entrambi appartengono a P o entrambi a D rappresenta una sottosequenza di A di somma pari. Inoltre, ogni sottosequenza $S(i, j)$ di somma pari e' rappresentata da una coppia (i, j) tale che $i, j \in P$ o $i, j \in D$.
Il numero totale di tali sottosequenze è quindi dato dal numero di coppie di elementi di somme prefisse con la stessa parita', ovvero:

$$\binom{|P|}{2} + \binom{|D|}{2} = \frac{|P|(|P|-1)}{2} + \frac{|D|(|D|-1)}{2}$$

Algorithm 4: PrefixPairs(A)

```
P ← 0 ;
D ← 0 ;
S ← PrefixSumBuild(A);
for i ← 0 to n do
    if S(i) è pari then
        P ← P + 1 ;
    else
        D ← D + 1 ;
return P(P−1)/2 + D(D−1)/2;
```

3.1 Correttezza

Lemma 3.1. *esistono due indici* $(i, j) : 0 \leq i < j \leq n$ *tali per cui* $S(j) - S(i)$ *è pari* $\iff$ *esiste una sottosequenza in* A *con somma degli elementi pari, rappresentata da* $(\alpha, \beta) : 1 \leq \alpha \leq \beta \leq n$.

Proof. Viene dimostrata prima $\implies$: Se esistono due indici $(i, j) : 0 \leq i < j \leq n$ tali per cui $S(j) - S(i)$ è pari, allora la sottosequenza rappresentata da $(i + 1, j)$ ha somma $\sum_{k=i+1}^{j} a_k = S(j) - S(i)$, la quale è pari per ipotesi. Si osservi che $1 \leq \alpha = i + 1 \leq \beta = j \leq n$.

Dunque viene dimostrata $\impliedby$. Se esiste una sottosequenza in A con somma degli elementi pari, rappresentata da $(\alpha, \beta) : 1 \leq \alpha \leq \beta \leq n$, allora da ipotesi $S(\beta) - S(\alpha - 1) = \sum_{k=\alpha}^{\beta} a_k$ è pari. Si osservi che $0 < i = \alpha - 1 \leq j = \beta \leq n$.

□

L'algoritmo si limita a contare il numero di coppie di indici (i, j) tali che $i, j \in P \vee i, j \in D$. Dal Lemma 3.1 questo è uguale al numero di sottosequenze la cui somma degli elementi e' pari.

3.2 Complessita' Computazionale

Theorem 3.2. *L'algoritmo* `PrefixPairs` *ha complessità computazionale superiormente limitata da una funzione in* $\Theta(n)$, *dove* $|A| = n$.

Proof. Dal Lemma 2.1, la costruzione della sequenza rappresentante le somme prefisse richiede $O(n)$ operazioni. Anche il conteggio del numero di elementi pari e dispari eseguito dall'`if` a riga 5 richiede $\Theta(n)$ operazioni. Infine, il calcolo della formula chiusa $\frac{P(P-1)}{2} + \frac{D(D-1)}{2}$ viene effettuato in tempo $O(1)$. Pertanto, la complessità temporale dell'algoritmo è $\Theta(n)$. □

3.3 Lower Bound

Qualsiasi algoritmo che determini in maniera esatta il numero di sottosequenze aventi somma degli elementi pari, deve necessariamente esaminare almeno una volta l'intera sequenza in input. Di conseguenza, qualunque algoritmo della sucitata tipologia esegue almeno $\Omega(n)$ istruzioni.

4 Can we do better?

L'algoritmo `PrefixPairs` ha complessità temporale che eguaglia il limite inferiore rispetto al problema, come osservato precedentemente, quindi non è possibile eseguire meno istruzioni(asintoticamente).
Segue un'idea che tenta di abbassare la complessita' spaziale dell'algoritmo `PrefixPairs`, che utilizza $\Omega(n \log \sum_{i=1}^{n} a_i)$ bit per memorizzare la sequenza delle somme prefisse.
Idea: si osservi che il valore restituito dall'algoritmo `PrefixPairs` dipende in modo esclusivo dai valori contenuti nelle variabili P e D, che rappresentano rispettivamente il numero di elementi pari e dispari all'interno della sequenza di somme prefisse. Per calcolare $S(i)$ sono necessari unicamente i valori $S(i-1)$ e a_i, dunque non è richiesto mantenere in memoria tutte le $S(i) \quad \forall i \in \{0, \ldots, n\}$, ma per ogni iterazione basta mantenere $S(i)$ ed $S(i + 1)$, ergo la complessità spaziale scende ad $O(1)$ per ogni iterazione.
Di seguito un algoritmo che utilizza la precedente osservazione, mantenendo ad ogni iterazione, il conteggio degli elementi pari e dispari nella sequenza delle somme prefisse. Nota che il numero di elementi pari e' inizializzato ad 1 dato che il primo elemento di S e' uguale a 0, cioe' $S(0) = 0$.

Algorithm 5: Streaming(A)

```
1  S ← 0;
2  P ← 1 ;                                   // S(0) = 0, pari
3  D ← 0;
4  for i ← 1 to n do
5      S ← S + a_i;
6      if S è pari then
7          P ← P + 1;
8      else
9          D ← D + 1;
10 return P(P−1)/2 + D(D−1)/2;
```

4.1 Correttezza

Dato che, durante l'i-esima iterazione, nella condizione alla riga 6 la variabile S coincide con $S(i)$, la dimostrazione di correttezza dell'algoritmo `PrefixPairs` si applica anche all'algoritmo proposto.

4.2 Complessita' Spaziale

La complessità temporale è, come per l'algoritmo `PrefixPairs`, $\Theta(n)$.
Per quanto riguarda la complessità spaziale, si analizzano il numero massimo di bit necessari per ogni variabile utilizzata nell'algoritmo. Si noti che gli elementi a_i della sequenza vengono forniti in streaming, quindi non è necessario memorizzarli. Le variabili P e D sono sempre minori o uguali ad n dato che rappresentano il conteggio di un sottoinsieme di elementi di A, quindi richiedono al piu' $\Theta(\log_2 n)$ bit. Al termine dell'n-esima iterazione si ha $S = \sum_{i=1}^{n} a_i$. Dunque servono $\Theta\big(\log_2\big(\sum_{i=1}^{n} a_i\big)\big)$ bit per rappresentare S.
Combinando i contributi di tutte le variabili si ottiene la seguente complessità spaziale:

$$\Theta(\log_2 n) + \Theta(\log_2 n) + \Theta\big(\log_2\big(\sum_{i=1}^{n} a_i\big)\big) = \Theta\big(\log_2 n + \log_2 \Theta\big(\sum_{i=1}^{n} a_i\big)\big).$$

Osservazione: se $\sum_{i=1}^{n} a_i = \Omega(n)$ (caso tipico quando gli a_i non sono tutti molto piccoli), la componente $\Theta(\log_2 n)$ è asintoticamente irrilevante rispetto alla quantità $\Theta\big(\log_2(\sum_i a_i)\big)$ e quindi la complessità spaziale è $\Theta\big(\log_2\big(\sum_{i=1}^{n} a_i\big)\big)$.

5 Again, can we do better?

Si può migliorare la complessità spaziale? La maggior quantita' di memoria utilizzata dall'algoritmo `Streaming` è impiegata da S, che rappresenta $S(i)$ all'iterazione i. Tuttavia, l'algoritmo `Streaming` necessita unicamente della parità di $S(i)$, questa si può mantenere incrementalmente mediante l'operatore modulo: $S \bmod 2$.
Inoltre, per calcolare la parità di $S(i+1)$, si osserva che questa è data da $(S(i) \bmod 2 + a_i \bmod 2) \bmod 2$.
Come ulteriore ottimizzazione, il conteggio degli elementi dispari nella sequenze delle somme prefisse è ridondante, infatti D si ricava dalla seguente relazione: $P + D = n + 1$. Segue un algoritmo che implementa tutte queste accortezze:

Algorithm 6: Streaming++(A)

```
S ← 0;
P ← 1 ;                                   // S(0) = 0, pari
for i ← 1 to n do
    S ← (S + a_i mod 2) mod 2;
    if S = 0 then
        P ← P + 1;
return P(P−1)/2 + (n−P+1)(n−P)/2;
```

5.1 Correttezza

Durante l'$i$esima iterazione, S contiene la parita' di $S(i)$.
Viene codificata la parita' con 0 e la disparita' con 1.
Se $i = 0$, la parita' e' determinata correttamente dalla riga 1, che imposta $S = 0$.
Supponiamo che per $i - 1 > 0$, S determini la parita' di $S(i - 1)$ correttamente.
Nell'iterazione i, $S \leftarrow (S + a_i \bmod 2) \bmod 2 = (S(i-1) \bmod 2 + a_i \bmod 2) \bmod 2 = (S(i-1) + a_i) \bmod 2$, che e' la parita' di $S(i)$.

5.2 Analisi

La complessità temporale è, come per l'algoritmo `PrefixPairs`, $\Theta(n)$.
Come per l'algoritmo `Streaming`, gli elementi a_i non vengono memorizzati, dunque non richiedono l'utilizzo di memoria. La variabile P può raggiungere al più il valore n, quindi richiede $\Theta(\log_2 n)$ spazio; la variabile S invece può valere solo 0 o 1, ad indicare la parità di $S(i)$ durante l'i-esima iterazione, quindi richiede $O(1)$ memoria.
In conclusione, la complessità spaziale dell'algoritmo `Streaming++` e' $\Theta(\log_2 n)$.

6 Tabella riassuntiva

Algoritmo	**Tempo**	**Spazio**
BruteForce	$\Theta(n^3)$	$O\left(\log_2\left(\sum_{i=1}^{n} a_i\right)\right)$
BruteForce ++	$\Theta(n^2)$	$\Theta\left(n\log_2\left(\sum_{i=1}^{n} a_i\right)\right)$
PrefixPairs	$\Theta(n)$	$\Theta\left(n\log_2\left(\sum_{i=1}^{n} a_i\right)\right)$
Streaming	$\Theta(n)$	$\Theta\left(\log_2\left(\sum_{i=1}^{n} a_i\right)\right)$
Streaming++	$\Theta(n)$	$\Theta(\log_2 n)$

Figure 5: Una tabella che riporta la complessita' temporale e spaziale di ogni algoritmo presentato.